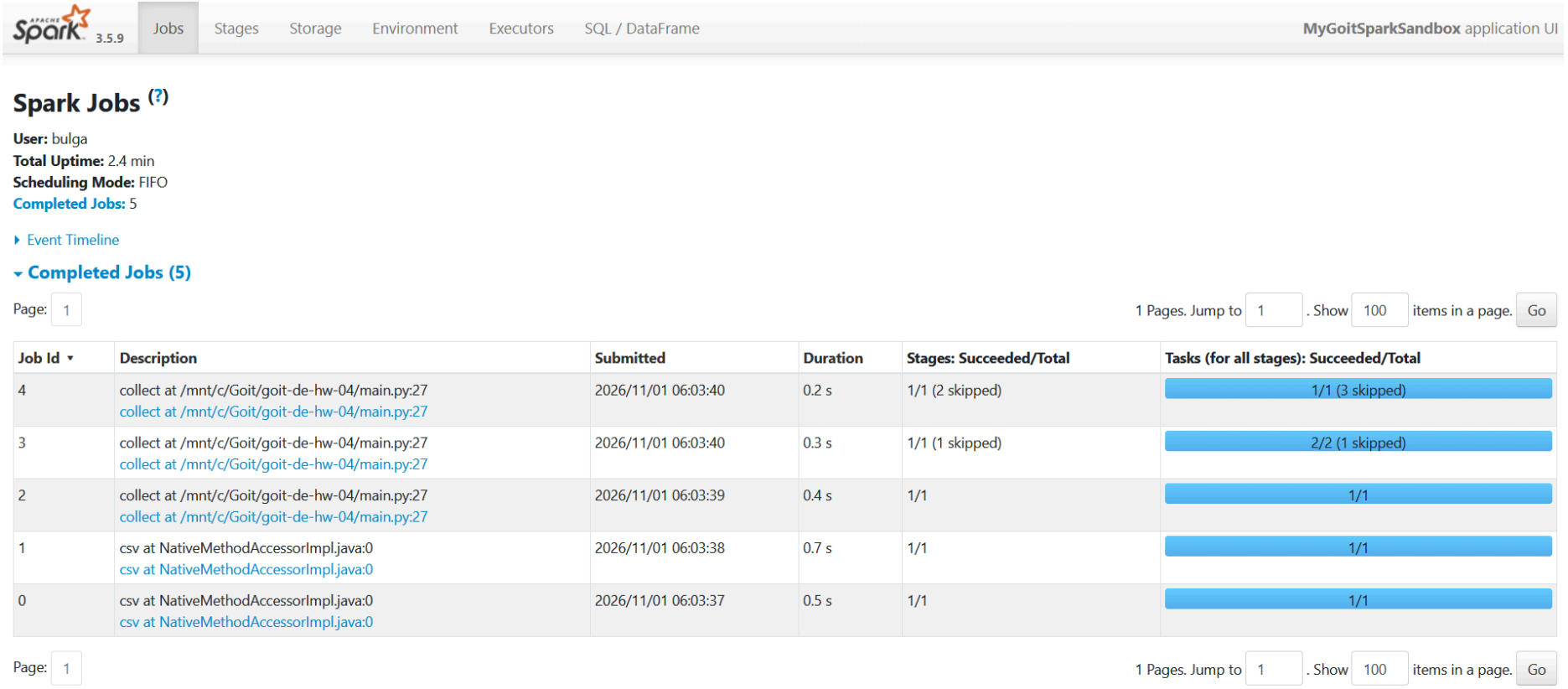
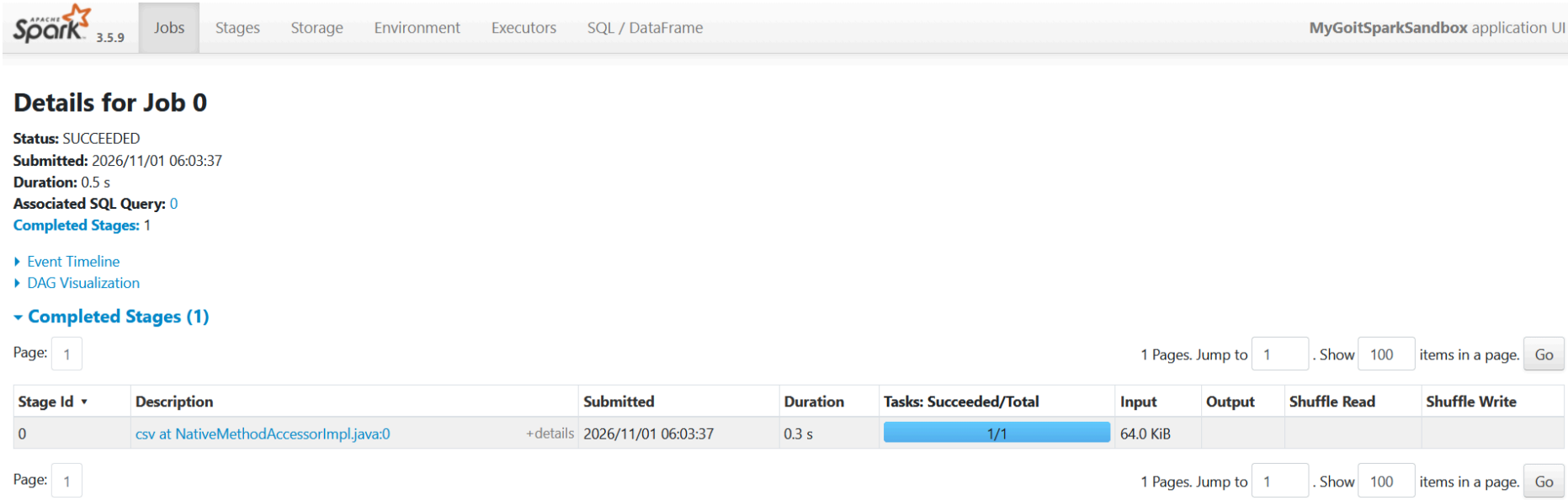
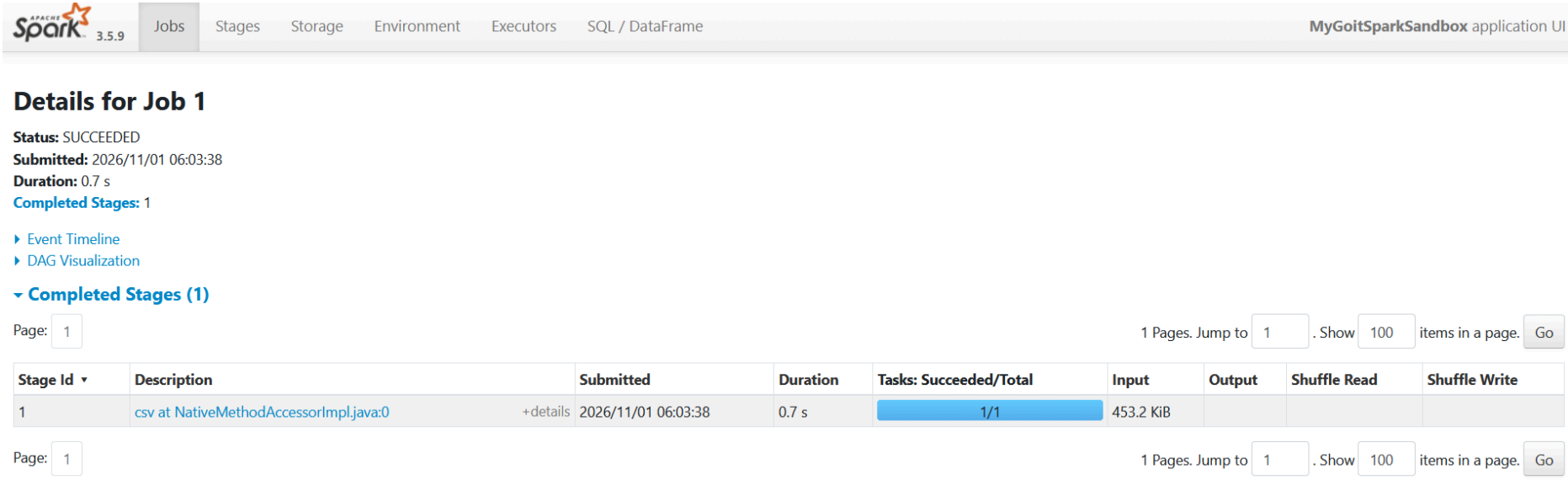
Part #1:


Spark 3.5.9
Jobs
Stages
Storage
Environment
Executors
SQL / DataFrame
MyGoitSparkSandbox application UI
Spark Jobs (?)
User: bulga
Total Uptime: 2.4 min
Scheduling Mode: FIFO
Completed Jobs: 5
Event Timeline
Completed Jobs (5)
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go
Job Id ▾ | Description | Submitted | Duration | Stages: Succeeded/Total | Tasks (for all stages): Succeeded/Total
4 | collect at /mnt/c/Goit/goit-de-hw-04/main.py:27 collect at /mnt/c/Goit/goit-de-hw-04/main.py:27 | 2026/11/01 06:03:40 | 0.2 s | 1/1 (2 skipped) | 1/1 (3 skipped)
3 | collect at /mnt/c/Goit/goit-de-hw-04/main.py:27 collect at /mnt/c/Goit/goit-de-hw-04/main.py:27 | 2026/11/01 06:03:40 | 0.3 s | 1/1 (1 skipped) | 2/2 (1 skipped)
2 | collect at /mnt/c/Goit/goit-de-hw-04/main.py:27 collect at /mnt/c/Goit/goit-de-hw-04/main.py:27 | 2026/11/01 06:03:39 | 0.4 s | 1/1 | 1/1
1 | csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0 | 2026/11/01 06:03:38 | 0.7 s | 1/1 | 1/1
0 | csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0 | 2026/11/01 06:03:37 | 0.5 s | 1/1 | 1/1
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go

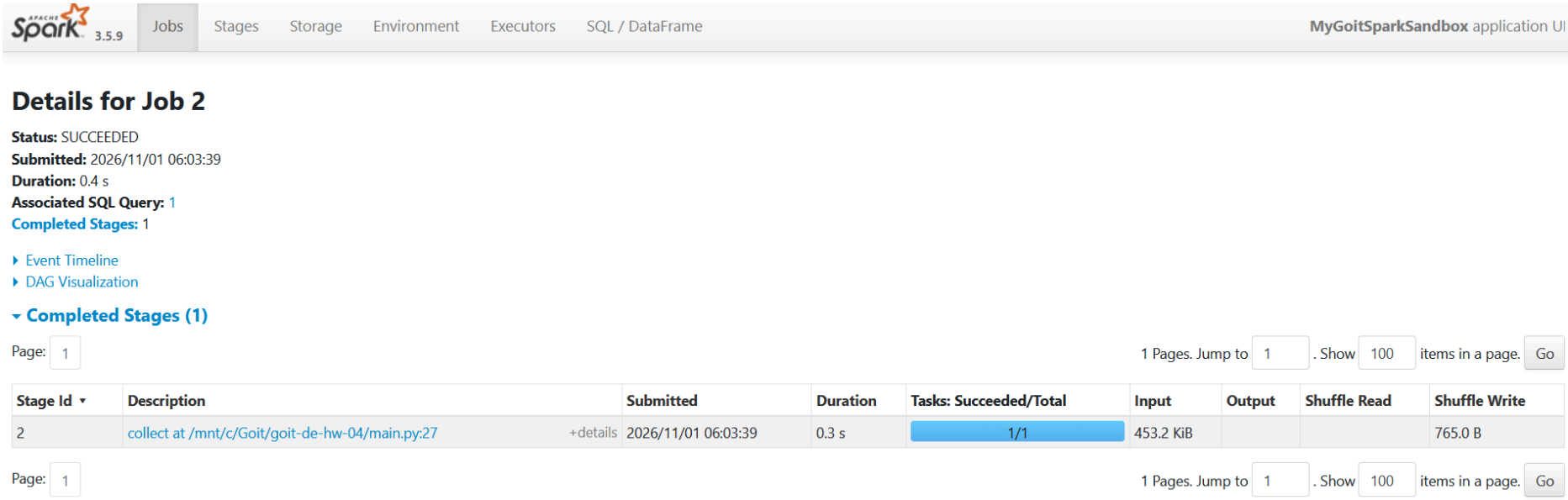
Job-0:


Spark 3.5.9
Jobs
Stages
Storage
Environment
Executors
SQL / DataFrame
MyGoitSparkSandbox application UI
Details for Job 0
Status: SUCCEEDED
Submitted: 2026/11/01 06:03:37
Duration: 0.5 s
Associated SQL Query: 0
Completed Stages: 1
Event Timeline
DAG Visualization
Completed Stages (1)
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go
Stage Id
Description
Submitted
Duration
Tasks: Succeeded/Total
Input
Output
Shuffle Read
Shuffle Write
0
csv at NativeMethodAccessorImpl.java:0
+details
2026/11/01 06:03:37
0.3 s
1/1
64.0 KiB
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go

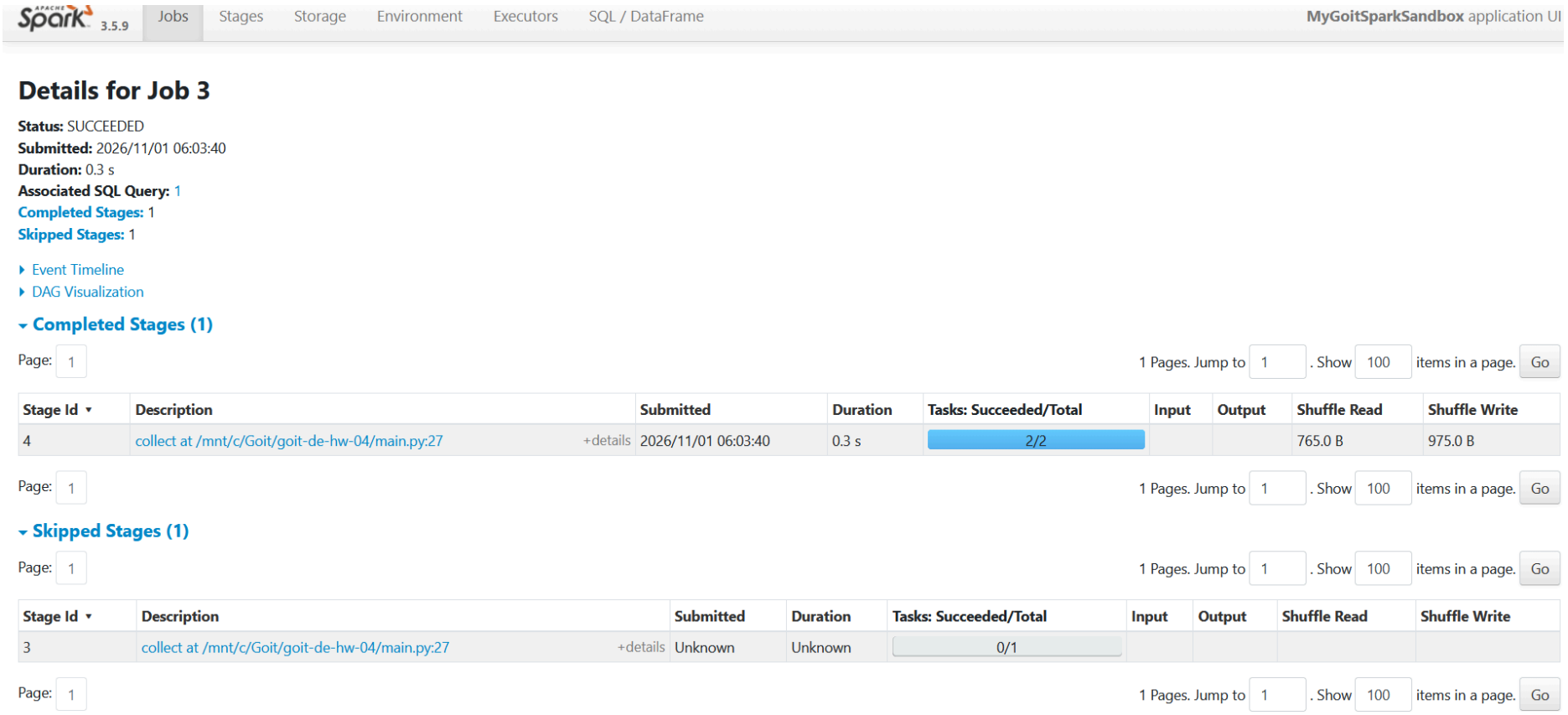
Job-1:


Spark 3.5.9
Jobs
Stages
Storage
Environment
Executors
SQL / DataFrame
MyGoitSparkSandbox application UI
Details for Job 1
Status: SUCCEEDED
Submitted: 2026/11/01 06:03:38
Duration: 0.7 s
Completed Stages: 1
Event Timeline
DAG Visualization
Completed Stages (1)
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go
Stage Id
Description
Submitted
Duration
Tasks: Succeeded/Total
Input
Output
Shuffle Read
Shuffle Write
1
csv at NativeMethodAccessorImpl.java:0
+details
2026/11/01 06:03:38
0.7 s
1/1
453.2 KiB
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go

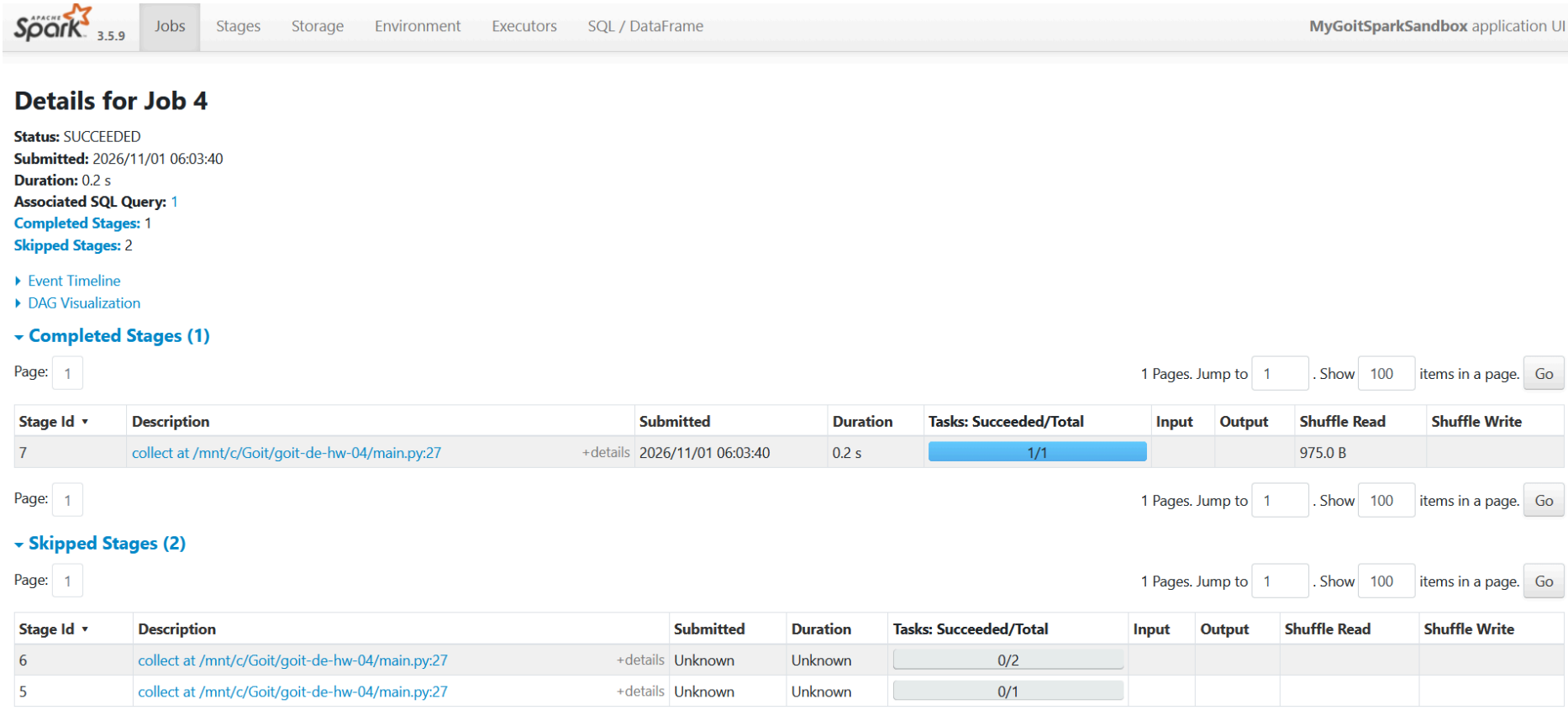
Job-2:


Spark 3.5.9
Jobs
Stages
Storage
Environment
Executors
SQL / DataFrame
MyGoitSparkSandbox application UI
Details for Job 2
Status: SUCCEEDED
Submitted: 2026/11/01 06:03:39
Duration: 0.4 s
Associated SQL Query: 1
Completed Stages: 1
Event Timeline
DAG Visualization
Completed Stages (1)
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go
Stage Id
Description
Submitted
Duration
Tasks: Succeeded/Total
Input
Output
Shuffle Read
Shuffle Write
2
collect at /mnt/c/Goit/goit-de-hw-04/main.py:27 +details
2026/11/01 06:03:39
0.3 s
1/1
453.2 KiB
765.0 B
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go

Job-3:


Spark 3.5.9
Jobs
Stages
Storage
Environment
Executors
SQL / DataFrame
MyGoitSparkSandbox application UI
Details for Job 3
Status: SUCCEEDED
Submitted: 2026/11/01 06:03:40
Duration: 0.3 s
Associated SQL Query: 1
Completed Stages: 1
Skipped Stages: 1
Event Timeline
DAG Visualization
Completed Stages (1)
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go
Stage Id
Description
Submitted
Duration
Tasks: Succeeded/Total
Input
Output
Shuffle Read
Shuffle Write
4
collect at /mnt/c/Goit/goit-de-hw-04/main.py:27 +details
2026/11/01 06:03:40
0.3 s
2/2
765.0 B
975.0 B
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go
Skipped Stages (1)
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go
Stage Id
Description
Submitted
Duration
Tasks: Succeeded/Total
Input
Output
Shuffle Read
Shuffle Write
3
collect at /mnt/c/Goit/goit-de-hw-04/main.py:27 +details
Unknown
Unknown
0/1
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go

Job-4:



Після запуску застосунку у Spark UI було зафіксовано 5 Jobs:
Job 0 та Job 1 були створені під час читання CSV-файлу із ввімкненим параметром inferSchema=true.
Job 2 був створений для виконання shuffle після операції repartition(2).
Job 3 був створений під час виконання агрегування groupBy().count().
Job 4 відповідає фінальному виконанню дії collect(), яка повертає результат драйверу.

Part #2:

Spark Jobs (?)

User: bulga
Total Uptime: 34 s
Scheduling Mode: FIFO
Completed Jobs: 8

▸ Event Timeline

▾ **Completed Jobs (8)**

Page: 1 — 1 Pages. Jump to 1 . Show 100 items in a page. Go

Job Id ▾	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
7	collect at /mnt/c/Goit/goit-de-hw-04/part2.py:30 collect at /mnt/c/Goit/goit-de-hw-04/part2.py:30	2026/11/01 22:30:18	95 ms	1/1 (2 skipped)	1/1 (3 skipped)
6	collect at /mnt/c/Goit/goit-de-hw-04/part2.py:30 collect at /mnt/c/Goit/goit-de-hw-04/part2.py:30	2026/11/01 22:30:18	0.2 s	1/1 (1 skipped)	2/2 (1 skipped)
5	collect at /mnt/c/Goit/goit-de-hw-04/part2.py:30 collect at /mnt/c/Goit/goit-de-hw-04/part2.py:30	2026/11/01 22:30:18	0.2 s	1/1	1/1
4	collect at /mnt/c/Goit/goit-de-hw-04/part2.py:25 collect at /mnt/c/Goit/goit-de-hw-04/part2.py:25	2026/11/01 22:30:17	0.1 s	1/1 (2 skipped)	1/1 (3 skipped)
3	collect at /mnt/c/Goit/goit-de-hw-04/part2.py:25 collect at /mnt/c/Goit/goit-de-hw-04/part2.py:25	2026/11/01 22:30:17	0.4 s	1/1 (1 skipped)	2/2 (1 skipped)
2	collect at /mnt/c/Goit/goit-de-hw-04/part2.py:25 collect at /mnt/c/Goit/goit-de-hw-04/part2.py:25	2026/11/01 22:30:16	0.2 s	1/1	1/1
1	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/11/01 22:30:15	0.9 s	1/1	1/1
0	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/11/01 22:30:14	0.4 s	1/1	1/1

Це схоже на типову поведінку Spark через lazy evaluation та те, що кожен action запускає окремий Job.

Spark будує свій план:

CSV Scan

↓

Repartition
↓
Filter
↓
Select
↓
GroupBy + Count
↓
Filter(count > 2)
↓
Collect

Чому не +1 Job, а +3?

Тому що collect() не дорівнює одному Job у фізичному виконанні.

У плані є дві shuffle-операції згідно коду:
repartition(2)
groupBy(...).count()

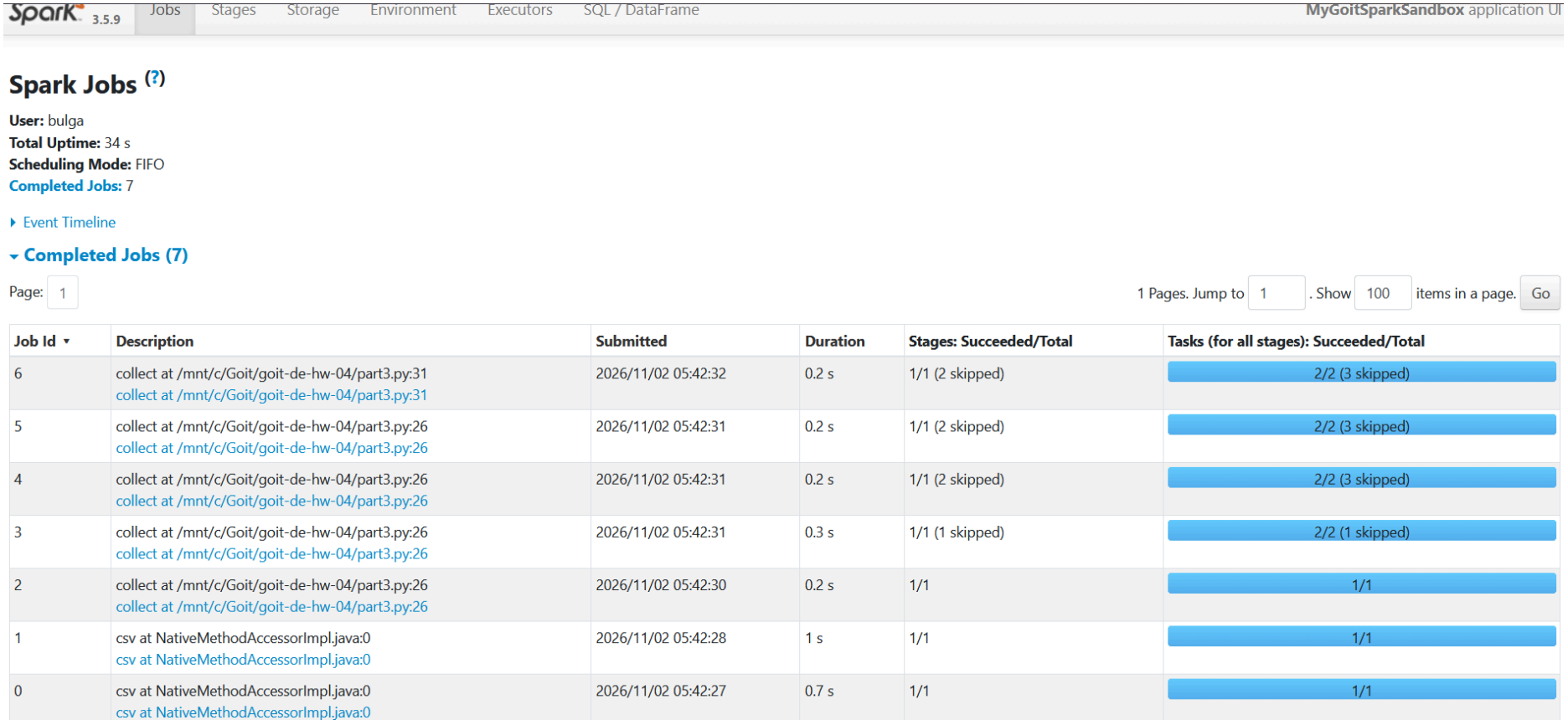
Кожна shuffle розбиває обчислення на декілька stage.

Коли викликається другий collect(), Spark повторно проходить через:

- читання CSV
- shuffle для repartition
- shuffle для groupBy
- фінальне збирання результату

Тому другий action породжує не один додатковий Job/Stage, а цілий набір робіт.

Part #3:



Job Id ▾	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
6	collect at /mnt/c/Goit/goit-de-hw-04/part3.py:31 collect at /mnt/c/Goit/goit-de-hw-04/part3.py:31	2026/11/02 05:42:32	0.2 s	1/1 (2 skipped)	2/2 (3 skipped)
5	collect at /mnt/c/Goit/goit-de-hw-04/part3.py:26 collect at /mnt/c/Goit/goit-de-hw-04/part3.py:26	2026/11/02 05:42:31	0.2 s	1/1 (2 skipped)	2/2 (3 skipped)
4	collect at /mnt/c/Goit/goit-de-hw-04/part3.py:26 collect at /mnt/c/Goit/goit-de-hw-04/part3.py:26	2026/11/02 05:42:31	0.2 s	1/1 (2 skipped)	2/2 (3 skipped)
3	collect at /mnt/c/Goit/goit-de-hw-04/part3.py:26 collect at /mnt/c/Goit/goit-de-hw-04/part3.py:26	2026/11/02 05:42:31	0.3 s	1/1 (1 skipped)	2/2 (1 skipped)
2	collect at /mnt/c/Goit/goit-de-hw-04/part3.py:26 collect at /mnt/c/Goit/goit-de-hw-04/part3.py:26	2026/11/02 05:42:30	0.2 s	1/1	1/1
1	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/11/02 05:42:28	1 s	1/1	1/1
0	csv at NativeMethodAccessorImpl.java:0 csv at NativeMethodAccessorImpl.java:0	2026/11/02 05:42:27	0.7 s	1/1	1/1

Тут cache() не зменшує кількість Action, але може зменшити кількість Job, які потрібні для повторного обчислення DAG.
Якщо дивитися на два collect(), то:

<u>Без cache()</u>

Перший collect() код:

nuek_processed.collect()

змушує Spark виконати весь ланцюжок:

CSV
↓
repartition
↓
filter
↓
select
↓
groupBy
↓
count
↓
collect

Потім виконується код:
nuek_processed = nuek_processed.where("count > 2")
nuek_processed.collect()

Оскільки результат попереднього collect() ніде не зберігався, Spark знову будує весь DAG з нуля:

CSV
↓
repartition
↓
filter
↓

select
↓
groupBy
↓
count
↓
filter(count > 2)
↓
collect

Тому CSV перечитується ще раз, shuffle виконується ще раз, агрегація виконується ще раз.

3 cache()

Тепер маємо код:
nuek_processed_cached = ...
.groupBy("unit_id")
.count()
.cache()

На цьому етапі ще нічого не кешується.
Spark лише запам'ятовує, коли вперше порахується цей DataFrame, то він збережу результат.

Після першого коду:
nuek_processed_cached.collect()

Spark:

читає CSV;
виконує repartition;

виконує groupBy;
отримує результат;
записує отримані дані в кеш.

У пам'яті тепер лежить результат:

Plain Text
(unit_id, count)

Коли далі виконується код:
nuek_processed = nuek_processed_cached.where("count > 2")
nuek_processed.collect()

Spark вже бачить, що батьківський DataFrame знаходиться в кеші.
Тому новий DAG виглядає приблизно так:

InMemoryTableScan
↓
filter(count > 2)
↓
collect

а не

CSV
↓
repartition
↓
groupBy
↓
...

Виходить, що кількість Job стало менше, бо Spark більше не виконує:

повторне читання CSV;
повторний repartition;
повторний shuffle для groupBy;
повторний count().

Він просто бере вже готові кешовані записи з пам'яті.